

KVFlow：通过高效前缀缓存加速 基于 LLM 的多智能体 workflow

Zaifeng Pan¹ Ajikumar Patel¹ Zhengding Hu^{1*} Yipeng Shen¹ Yue Guan¹
Wan-Lu Li¹ Lianhui Qin¹ Yida Wang² Yufei Ding¹

¹ UCSD ² AWS

摘要

基于大语言模型（LLM）的智能体 workflow，已成为协调多个专业智能体解决复杂任务的常用范式。为提高服务效率，现有系统使用前缀缓存，复用智能体固定提示词对应的键值（KV）张量，避免反复调用中的冗余计算。不过，系统通常采用最近最少使用（LRU）策略淘汰 KV 缓存，无法预见智能体未来的使用，常在即将复用前丢弃相关缓存，导致频繁未命中，以及大量重新计算或交换开销。我们提出 KVFlow，一个专门面向智能体工作负载、感知 workflow 的 KV 缓存管理框架。它将智能体执行顺序抽象为“智能体步数图”，为每个智能体设置距执行步数，估计其距未来激活的远近。这些值指导 KV 节点级的细粒度淘汰策略，使系统保留更快会被复用的条目，并有效管理树状缓存中的共享前缀。此外，KVFlow 提出完全重叠的 KV 预取机制，通过后台线程，将预计下一步使用的张量提前从 CPU 加载到 GPU，避免生成过程因缓存未命中而停顿。

相对采用分层基数树缓存的 SGLang，KVFlow 在大提示词单 workflow 场景中最多获得 1.83 $\times$ 加速，在多个并发 workflow 场景中最多获得 2.19 $\times$ 加速。

1 引言

LLM 智能体 workflow 协调多个专业智能体，每个智能体具有固定提示词，负责特定子任务，以模块化、可解释的方式求解复杂问题 [1–5]。例如，MetaGPT [3] 按产品经理、工程师等软件工程角色组织协作。这种设计改善复用性与一致性，但 workflow 需要反复为各智能体调用 LLM，也会产生较高推理延迟。

为缓解这些开销，现有智能体框架和应用 [3, 6–10] 依赖具备系统优化的 LLM 服务系统 [11–13]。常用技术是前缀缓存 [12, 14]：跨解码步骤与请求，复用自注意力层为静态提示词 token 产生的 KV 张量。它尤其适合智能体 workflow，因为每个智能体都有固定提示词，描述名称、职责和行为特征。这些内容跨迭代不变，前缀缓存能避免重复计算静态内容，显著缩短每个智能体的推理时间。

但 GPU 内存有限时，仅有前缀缓存仍不够。现有系统通常使用 LRU，淘汰最近未被访问的 KV 缓存。我们观察到，它在智能体 workflow 中可能表现不佳。图 1 的示例 workflow [5]，把四个智能体组成反复执行的顺序管线。Executor 执行时，LRU 认为 Expresser 的 KV 缓存最近未被访问，应当淘汰；然而 workflow 马上就会进入 Expresser，因此很快又未命中。这种淘汰引入不必要的重新计算，降低整体执行效率。

为解决这些限制，我们提出感知 workflow 的 KV 缓存管理框架 KVFlow。首先，引入智能体步数图这一灵活抽象，捕捉执行依赖，并支持条件分支和同步屏障等不同结构。每个节点带有计算得到的距执行步数，估计智能体还有多久会运行；步数聚合函数沿图传播这些值，使系统能够理解动态、具有结构的执行模式。

*通讯作者

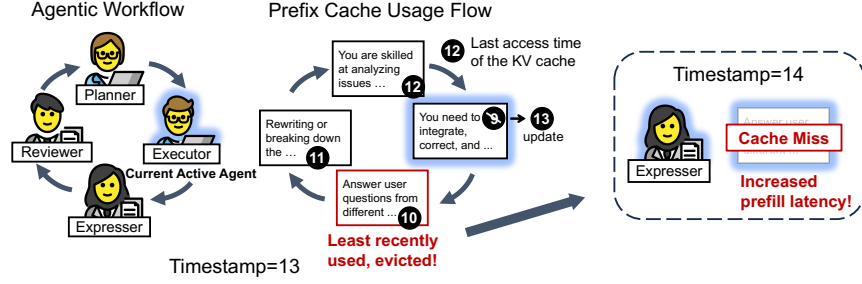


图 1: 改编自文献 [5] 的循环工作流, 包含 Planner (规划)、Executor (执行)、Expresser (表达) 和 Reviewer (审查) 四个智能体。时间戳 13, Executor 活跃并更新缓存, LRU 淘汰 Expresser 的缓存; 时间戳 14, Expresser 再次执行时未命中, 预填充时间增加。图中 Agentic Workflow 为智能体工作流, Prefix Cache Usage Flow 为前缀缓存使用过程, Current Active Agent 为当前活动智能体, Last access time 为缓存最后访问时刻, update 为更新; 红字分别表示“最近最少使用, 已淘汰”“缓存未命中”“预填充延迟增加”。提示词片段依次表示“你擅长分析问题……”“你需要整合、校正并……”“从不同……回答用户问题”“重写或分解……”。

运行时, 系统以两种方式利用这些信息优化缓存。第一, 采用感知工作流的淘汰策略, 优先淘汰距执行步数较大的智能体缓存。多个智能体可通过树状缓存共享公共前缀, 因此我们进一步在缓存节点级设置优先级, 实现细粒度管理。第二, 根据步数图预测下一批将调用的智能体, 提前将其 KV 张量从 CPU 加载到 GPU, 实现完全重叠的预取, 在不阻塞生成的情况下消除前缀缓存未命中开销。两项优化共同提高缓存效率, 缩短工作流延迟。

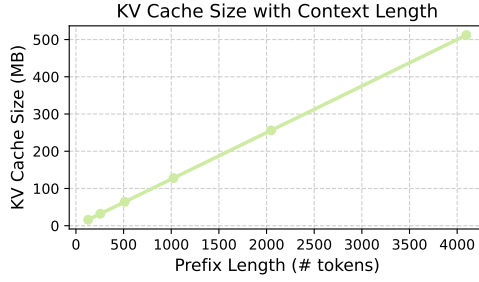
本文贡献如下:

- 指出现有服务系统的一项基础性低效: 广泛采用的 LRU 缓存淘汰策略, 在智能体工作流中可能产生次优性能。
- 提出感知工作流的缓存管理优化 KVFlow, 按智能体执行顺序设置淘汰优先级, 并通过完全重叠的预取消除未命中开销。
- 全面评估系统, 结果显示它显著减少未命中开销: 相对采用分层基数树缓存的 SGLang, 在大提示词单工作流与多个并发工作流场景中, 分别最多获得 $1.83\times$ 与 $2.19\times$ 加速。

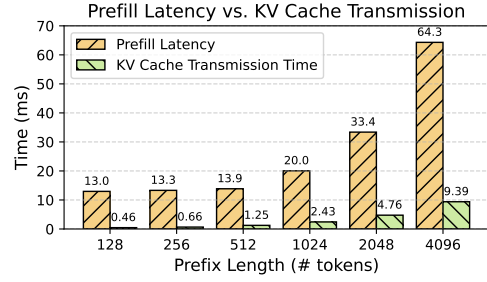
2 背景

LLM 服务系统中的前缀缓存。 为细粒度复用前缀并避免冗余存储, 现代服务系统 [11, 12] 在 GPU 上以树结构组织 KV 缓存, 每个节点保存一段 token 及其对应 KV 张量。新请求到达时, 从根节点匹配前缀, 再连接匹配路径上的张量, 重构完整缓存前缀。GPU 内存不足时, 系统按 LRU 淘汰节点。内存耗尽有两种常见原因: 大量并发用户分别执行不同工作流, 形成许多活动缓存条目; 或者智能体提示词很长, 而硬件容量有限。图 2 (a) 显示, 单请求的缓存大小随前缀长度快速增长, 长上下文带来更大内存压力。CPU 内存还可作为第二级缓存, 备份被淘汰的张量, 并通过 PCIe 进行交换。尽管存在 PCIe 延迟, 交换通常仍明显快于重新计算 [15, 16]。图 2 (b) 比较 PCIe 传输与预填充计算时间, 表明内存压力下卸载至 CPU 是有效策略。

智能体工作流。 智能体工作流 [8, 10, 9, 7] 将多个智能体组织成执行图, 协同完成复杂任务。相对完全自主的智能体 [17–19], 它利用人的领域知识, 使不同任务上的性能更一致、更稳健 [3, 20–24]。每个智能体通常发起一次或多次调用, 提示词由固定部分与任务特定的动态部分组成。固定部分编码角色、行为指令、任务说明和少样本示例, 可能很长。例如, 文献 [22] 的 TestBench Agent 与 RTL Generator Agent, 固定提示词分别包含超过 3000 和 1000 个 token 的详细少样本示例。因此, 缓存固定部分的 KV 能显著缩短预填充, 改善工作流效率。动态部分通常包含用户问题或指令, 缓存价值较低。

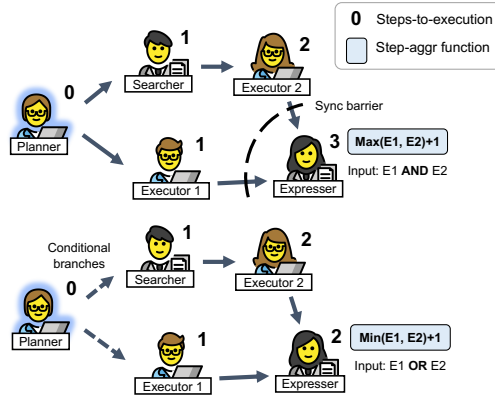


(a) Llama-3.1-8B 的 KV 缓存大小

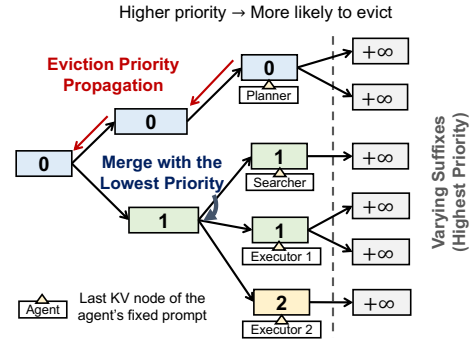


(b) Llama-3.1-8B 的预填充与 PCIe 缓存传输耗时，批量为 1

图 2: 不同上下文长度下的 KV 缓存特征。(a) 缓存大小随 token 数线性增加，内存使用随之上升；横轴为前缀 token 数，纵轴为缓存大小 (MB)。(b) PCIe 传输明显快于重新计算；横轴为前缀长度，纵轴为耗时 (毫秒)，图例为预填充延迟和 KV 缓存传输时间。



(a) 两个不同步数图中，各智能体的距执行步数



(b) 缓存树中各 KV 节点的淘汰优先级，多个智能体可共享部分前缀

图 3: 感知工作流的淘汰策略。(a) 将工作流抽象为智能体步数图，通过依赖边上的步数聚合函数计算距执行步数。Planner、Searcher、Executor、Expresser 分别为规划、搜索、执行和表达智能体；Sync barrier 为同步屏障，Conditional branches 为条件分支；E1 AND E2 要求两个执行器都完成，E1 OR E2 允许任一个完成；Max / Min 分别取最大 / 最小值。(b) 把步数沿缓存树传播为节点淘汰优先级；越小越应保留，减少过早淘汰。图中 Eviction Priority Propagation 为优先级传播，Merge with the Lowest Priority 为按最小值合并，Last KV node 为固定提示词末尾节点；Varying Suffixes 为变化后缀，赋予最高淘汰优先级。数值越大，越容易淘汰；正无穷表示首先考虑淘汰。

3 KVFlow 的设计

本节介绍 KVFlow 的设计，通过两项技术改善智能体工作流的前缀缓存管理。第一，感知工作流的淘汰策略，按未来使用情况设置 KV 节点优先级，改善默认 LRU。第二，重叠 KV 预取机制，通过提前加载和感知状态的调度，隐藏 CPU-GPU 传输延迟。

3.1 感知工作流的淘汰策略

现有系统通常采用 LRU，但它在智能体工作流中可能不够理想。即将执行的智能体可能已经空闲很久，而刚执行完的智能体近未来未必会再次使用。此外，最近执行的智能体动态生成的后缀，会随任务进展快速变化，通常不会复用，却仍被暂时保留。利用工作流信息，可以预测即将执行的智能体序列，作出更合理的淘汰决策。

智能体步数图与距执行步数。 为按工作流结构淘汰缓存，需要先捕捉智能体的依赖关系。但真实工作流的交互非常多样。图 3 (a) 中，上方工作流的 Expresser 依赖 Executor1 与

Executor2 同时完成；下方包含条件分支，任一 Executor 完成都可能触发 Expresser。传统控制流图（CFG）或 DAG 抽象，难以统一表达这些不同执行语义。

为此，我们引入智能体步数图：每个节点对应一次智能体调用，边表示依赖关系。与普通图不同，每个节点还有一个步数聚合函数，决定如何从前驱推导其距执行步数。对于前缀缓存管理，我们只关注每个智能体最早可能执行的步骤，抽象掉依赖的具体类型。

例如，图 3 (a) 上方工作流要求两个执行智能体都完成，因此 Expresser 的步数为 $\max(E1, E2) + 1$ ；下方任一路径即可触发，因此为 $\min(E1, E2) + 1$ 。递归应用这些聚合函数，就能统一计算多智能体工作流中的距执行步数。²

感知工作流的淘汰优先级设置。 根据步数图中的距执行步数，我们为各 KV 缓存节点设置细粒度淘汰优先级。图 3 (b) 中，距执行步数较大的智能体，其缓存更可能被淘汰。由于不同智能体可在树状缓存中共享公共前缀，优先级按缓存节点设置，而非只按整个智能体设置。

具体而言，只有固定提示词部分按距执行步数设置优先级；全部变化后缀始终设为最高淘汰优先级，便于尽早释放。每个智能体将其步数赋给固定提示词的最后节点，再沿树向上传播。当一个节点汇集多个智能体的信息时，取子节点中最小的优先级，即最不应淘汰的值，保证只要任何智能体近期仍需要该共享节点，就尽量保留它。

这一传播方式生成随工作流变化的缓存优先级映射，反映潜在复用机会。GPU 内存紧张时，系统先淘汰变化后缀，再按优先级从大到小逐步淘汰前缀节点，优先删除近期不太会复用的内容。多个并发工作流也适用；共享节点的冲突，以所有工作流中最低、最保守的淘汰优先级解决。

3.2 重叠 KV 预取

感知工作流的淘汰避免过早删除即将使用的缓存，但被淘汰的智能体再次执行时，仍可能未命中。长提示词尤其昂贵，因为从头重新计算 KV 需要大量时间。因此，我们使用 CPU 作为第二级缓存，保存被淘汰智能体固定提示词的 KV。

已有系统在配置 CPU 缓存后，通常采用被动加载，如图 4 顶部时间线。Executor 1 的缓存卸载到 CPU 后，直到调度该智能体时才重新加载，虽然避免重新计算，但 CPU 到 GPU 的传输仍增加明显延迟，尤其是长前缀。

主动预取。 我们利用工作流信息，提前异步加载所需 KV，形成主动预取。图 4 第二条时间线中，Planner 执行时，系统预期下一步调用 Executor 1，便提前从 CPU 向 GPU 预取其缓存。智能体执行主要包括 GPU 上的模型前向计算与下一 token 采样，采样时模型输出从 GPU 传到 CPU；KV 加载则是 CPU 到 GPU 的反方向传输。两类操作使用不同硬件资源，可以并行。PCIe 的全双工能力允许两个方向同时通信。当工作流包含分支时，在预设的并发预取数量限制内，系统保守地预取步数图中所有可能下一步执行的智能体。³

但单独预取并不总是足够。如果当前智能体的执行时间短于预取耗时，后续生成仍会被未完成的加载阻塞。高并发时，多个工作流争用 CPU-GPU 带宽，产生排队延迟，这种情况更常见。图 4 第二条时间线中，Executor 1 即使已开始预取，生成仍出现停顿。

感知状态的调度。 为进一步减少 GPU 空闲，每个调度步骤都会检查缓存状态。若请求的前缀仍在加载，暂时跳过它，优先执行其他已就绪请求，例如图 4 的 Executor 2，或其他并发工作流的请求。每个缓存节点关联四种状态之一：位于 GPU、在 CPU 中备份、加载中或卸载中。调度器检查请求所需的全部节点，跳过加载中的节点，避免重复加载；所有依赖可用后才派发请求。后台加载线程完成后更新节点状态，通知调度器已就绪。卸载中的节点同样不参与淘汰决策，避免回收内存时产生竞态。

²译注：步数衡量执行次序上的距离，不是实际秒数或剩余执行时间分布；min 表示最早可能触发，未按分支发生概率计算期望，也没有为全部分支建立概率模型。

³译注：全双工支持双向并行，不意味着任意并发负载下都不存在带宽、内存或调度竞争；下文也明确讨论多个工作流争用传输资源而造成预取延迟。

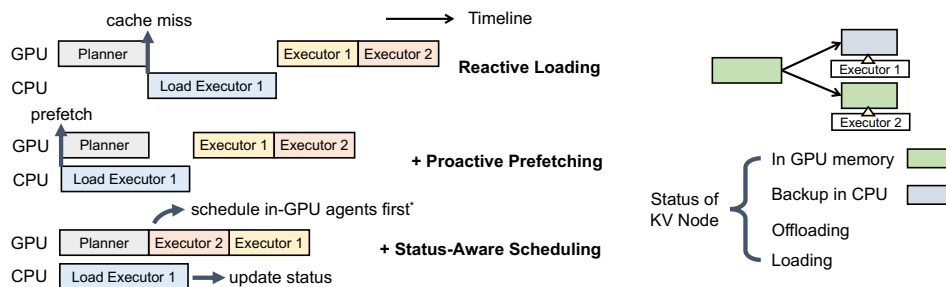


图 4: 重叠 KV 预取。相对被动加载，系统结合提前加载下一步缓存的主动预取，以及感知状态的调度，减少 CPU-GPU 传输开销。三条时间线依次为 Reactive Loading（被动加载）、Proactive Prefetching（主动预取）、Status-Aware Scheduling（状态调度）；Load Executor 1 为加载执行器 1，cache miss 为未命中，prefetch 为预取，update status 为更新状态，schedule in-GPU agents first* 为先调度缓存已在 GPU 的智能体。四种节点状态为位于 GPU、在 CPU 中备份、卸载中、加载中。星号说明：已在 GPU 的智能体既可来自同一工作流，也可来自其他并发工作流。

图 4 第三条时间线中，主动预取与状态调度结合，系统有效消除缓存未命中，使 GPU 计算与预取完全重叠，从而隐藏 CPU-GPU 传输延迟。⁴

3.3 实现

原型基于 SGLang v0.4.4 [12]，该服务系统同时提供执行后端与应用开发前端。其后端使用基数树管理前缀缓存；我们扩展该机制，加入感知工作流的淘汰与完全重叠的预取。同时修改前后端，传输工作流信息。当前原型集成到 SGLang 前端 API，但方法不限于它：修改其他框架发送到服务端的 HTTP 请求，也可适配该方法。

步数信息获取。 运行时优化需要获得步数图产生的距执行步数。实现假设每个 `sgl.function` 对应一个独立智能体。执行时，即时替换 LLM 调用，将工作流元数据嵌入 HTTP 请求，其中包含当前智能体身份，以及步数图中所有智能体的距执行步数，指示后续可能调用的智能体。后端收到后更新缓存树的淘汰优先级；如果 GPU 可淘汰内存足够，就触发预取。

除步数图拓扑外，还需要跟踪各智能体固定提示词的末尾 KV 节点，如图 3。区分请求的固定部分与动态部分并不容易，因此提供两种方法：其一是显式原语接口，让用户标记固定部分结束位置；其二是启发式方法，跟踪智能体缓存命中历史，将持续命中的前缀视为固定部分。

客户端跟踪。 多个工作流可能并发使用同一后端。现有系统不区分请求来源，可能产生命名冲突，例如两个工作流都定义为“Planner”的智能体。因此，我们为每个应用分配唯一客户端 ID，并附加到每个后端请求中，区分智能体身份，避免不同客户端的工作流相互干扰。

4 评估

我们通过多组微基准，考察不同缓存和执行条件下的性能，主要回答两个问题：（1）提示词前缀较长、GPU 内存有限时，能否降低单工作流端到端延迟？（2）多个工作流并行的高并发场景中表现如何？我们先分析单工作流延迟，见第 4.1 节，再研究多工作流执行，见第 4.2 节。

⁴译注：状态调度通过改为执行已就绪请求，隐藏尚未完成的传输；如果没有其他可运行请求，或内存与带宽条件不足，不能保证任意场景都完全消除停顿。缓存仍可能未命中，所隐藏的是相关等待开销。

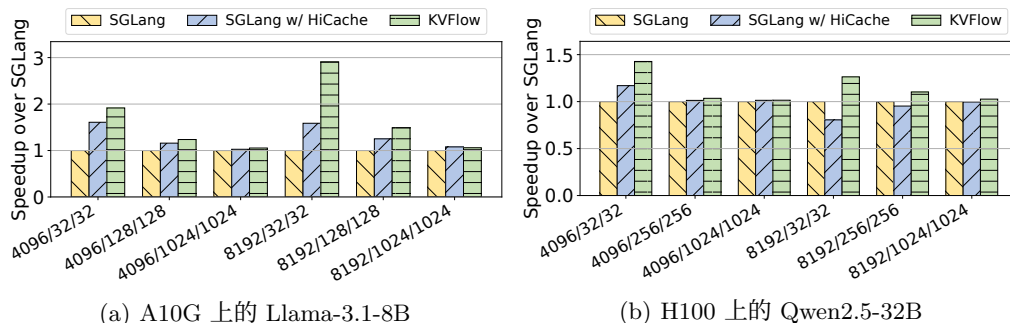


图 5: 10 智能体顺序工作流，相对仅 GPU 缓存版 SGLang 的加速。横轴依次为固定部分 *token* 数/动态部分 *token* 数/输出 *token* 数，纵轴为加速倍数。图例比较 SGLang、带 HiCache 的 SGLang 和 KVFlow；所有柱均以 SGLang 为 1，因此 KVFlow 与 HiCache 之间的加速应取两柱高度之比。

KVFlow 只修改系统缓存管理，不改变模型权重、提示词或解码逻辑，因此保持原有输出语义。实验只关注延迟等系统性能指标。⁵

4.1 单工作流延迟

首先评估批量为 1 时，执行单个智能体工作流的延迟。这对用户在 notebook 或开发工具中单独触发工作流的交互使用。与通过批处理提高吞吐量的在线服务相比，这类场景更重视单个请求的响应速度。

基准为包含 10 个智能体的顺序工作流。如第 2 节所述，每个输入由跨调用共享的固定前缀，以及随运行变化的动态后缀组成。我们通过随机采样 *token* 序列生成合成提示词，控制两部分长度。

模型与平台。 使用两组配置：(1) NVIDIA A10G 上的 Llama-3.1-8B，GPU 内存 24 GB，PCIe Gen1 带宽 2 GB/s；(2) NVIDIA H100 上的 Qwen2.5-32B，内存 80 GB，PCIe Gen5 带宽 64 GB/s。Llama 使用 32 个注意力头、8 个 KV 头，Qwen 使用 40 个注意力头、8 个 KV 头。为稳定测量延迟，使用确定性解码，即温度为 0、贪心采样。这两种配置代表 GPU 内存较紧张的场景。

基线。 比较两种 SGLang 配置。第一种记为 **SGLang**，仅在 GPU 保存基数树 KV 缓存，没有 CPU 备份；GPU 内存不足时淘汰节点，再次使用必须从头重新计算。第二种记为 **SGLang w/ HiCache**，启用 SGLang 默认的 CPU 缓存扩展，即分层基数树缓存，异步备份常用节点到主机内存。节点被淘汰后再次访问时，从 CPU 加载而非重新计算。为降低传输成本，HiCache 将第 l 层 GPU 计算与第 $l+1$ 层加载重叠，形成简单两级流水线。

评估方法。 先多次执行各智能体固定提示词，预热缓存，保证前缀已建立；HiCache 配置还会完成 CPU 备份。随后执行 10 智能体工作流十次，每次使用不同动态后缀，对延迟取平均。这模拟工作流反复调用或循环执行的使用模式 [5]。

结果。 图 5 给出不同提示词配置相对 GPU 缓存版 SGLang 的加速，配置记为固定部分/动态部分/输出。我们特意使用较长固定前缀，例如 8192 个 *token*，使缓存需求超过 GPU 容量，强制发生淘汰。

所有设置中，KVFlow 都获得最高加速。例如 A10G 的 8192/32/32 配置下，相对 HiCache 加速 1.83 $\times$ ，相对仅 GPU 缓存的 SGLang 加速 2.91 $\times$ 。这说明感知工作流的淘汰与预取，有效隐藏 CPU-GPU 传输开销。HiCache 虽能减少重新计算，但仍受流水线冷启动，以及计算短于传输时重叠不足的影响。

⁵译注：这里所称“保持语义正确性”，是指缓存管理优化不应改变原有模型计算语义，并不保证 LLM 回答本身对任务正确；本文没有把任务准确率提升作为实验证据。

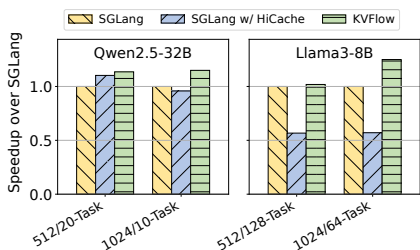


图 6: H100 上, 不同固定提示词长度/并发量下的高并发性能。左图为 Qwen2.5-32B, 右图为 Llama3-8B; 横轴如 1024/64-Task 表示每个智能体固定长度为 1024, 并发 64 个工作流; 纵轴为相对 SGLang 的加速。图例比较 SGLang、HiCache 与 KVFlow。

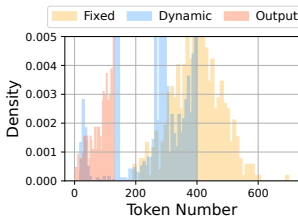


图 7: PEER 式工作流的 token 长度分布。横轴为 token 数, 纵轴为密度; Fixed、Dynamic、Output 分别为固定提示词、动态提示词与输出。

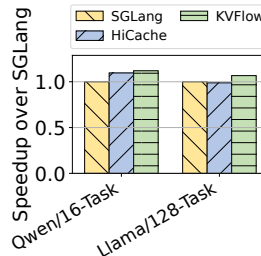


图 8: PEER 式多智能体应用上的加速比较。纵轴为相对 SGLang 的加速, 图例为 SGLang、HiCache 与 KVFlow; 横轴分别为 Qwen 下 16 个并发工作流, 以及 Llama 下 128 个并发工作流。

总体上, HiCache 通常优于仅 GPU 缓存, 因为从 CPU 加载往往快于重新计算。但 H100 的部分长上下文配置, 例如 8192/32/32, 中, HiCache 收益很小, 甚至更慢。可能原因是 SGLang 流水线调度不够理想: 内存争用或传输量较大时, CPU-GPU 传输没有充分重叠。

输出 token 数增加时, 系统相对收益减小, 因为解码主导总耗时, 缓存加载占比下降。已有很多研究优化自回归解码, 例如推测解码 [25–27]、KV 缓存稀疏化 [28] 和提前退出 [29], 与本文方法互补, 可共同使用。

固定提示词越长, 系统加速越明显: 固定长度为 8192 时平均加速为 1.48 $\times$, 固定长度为 4096 时为 1.28 $\times$ 。前缀越长, 未命中成本越大, 主动缓存管理收益也越高。

4.2 高并发工作流性能

我们在单张 H100 同时启动多个独立工作流, 进一步考察高并发性能。假设工作流互不交互、互不共享。图 6 比较四种配置, 标注每个智能体的固定提示词长度及并发工作流数量; 动态和输出长度均固定为 256。每组选择适当并发量, 保证 GPU 在支持活动请求后, 仍有内存用于前缀缓存。如果并发过高, 活动请求耗尽全部内存, 系统便无法保留可复用前缀, 这已超出本文优化范围。

结果。 图 6 中, 系统在各设置下都优于 SGLang 和 HiCache, 相对 SGLang 最高获得 1.25 $\times$ 加速。固定长度 1024 的收益高于 512, 因为前者未命中开销更大。HiCache 在高并发下表现较差, 多个配置甚至不如 SGLang。例如固定长度 1024、并发 64 个工作流时, 它的性能只有未使用 CPU 缓存的 SGLang 的 0.57 $\times$ 。我们推测, 频繁未命中触发被动回载, 扰乱了 SGLang 的调度与计算流水线; KV 存储碎片化也妨碍 PCIe 带宽充分利用。KVFlow 同样没有解决碎片问题, 但通过更合理淘汰与主动预取, 更好地重叠传输和计算。因此, 相对朴素 LRU 与被动加载的 HiCache, 最高获得 2.19 $\times$ 性能提升。

接近真实场景的工作流模拟。 我们基于 PEER [5] 框架模拟部署。每个工作流包含四个智能体, 使用 PEER 提供的模板实例化。为每个智能体采样角色和指令, 再让 LLM 生成提示词。采样的随机性使相似角色与指令也会产生不同提示词; 同一应用上下文又使它们经常部分共享前缀。这样得到既多样、又部分重复的工作流, 反映多智能体应用的常见特征。

使用 PEER 的 Financial QA 数据集作为工作流输入。负载规模适中, 智能体提示词通常为数十至数百个 token。图 7 展示固定、动态与输出部分的长度分布。图 8 比较 KVFlow、SGLang 与 HiCache: 相对后两者, 分别最多获得 1.12 $\times$ 与 1.08 $\times$ 加速, 说明该方法在接近真实部署的多应用服务中具有应用潜力。

5 相关工作

LLM 服务优化。 一类研究通过请求调度改善在线服务，包括连续批处理，亦称迭代级调度 [30]；使用多级反馈队列缓解队头阻塞 [31]；面向流式场景、感知体验质量的调度器 [32] 等。另一类研究关注 KV 管理。vLLM 的 PagedAttention [11] 通过分页存储减少碎片，SGLang 的 RadixAttention [12] 消除前缀缓存冗余。还有专门针对聊天机器人的缓存策略 [16, 33]。InferCept [34] 预测工具调用时长，再用成本模型决定保留、交换或丢弃外部中断请求的 KV。这些优化与本文方法互补，KVFlow 专注于多个智能体组成的工作流。

Autellix [35] 和 ParrotServe [36] 研究工作流请求调度，本文将它们描述为未考虑前缀缓存管理，认为研究目标互补。⁶

智能体工作流框架。 近期多智能体框架 [3, 4, 6, 7, 37–39]，以结构化角色组织智能体协同求解。它们提供消息传递与依赖构建机制，便于将工具和推理方法集成到智能体行为中，并提供常见角色与行为抽象，以及支持并发协作的多线程执行。部分系统把工作流抽象为计算图，节点是调用 LLM 的智能体，边是控制流或消息依赖。据此可通过剪边 [40]、插入算子 [8] 或优化拓扑 [7, 10]，改善应用正确性或质量。不过，这些框架主要构建应用层逻辑，生成仍依赖常规服务基础设施。本文利用工作流结构优化服务系统本身，面向多智能体执行时的后端效率。

6 结论

我们提出 KVFlow，一个感知工作流的 KV 缓存管理框架，优化智能体工作流中的 LLM 服务。它将执行抽象为步数图，计算每个智能体距执行的步数，形成预见未来使用的淘汰策略；还通过完全重叠的预取，主动消除未命中造成的停顿。实验显示，它在长提示词或高并发工作流中，显著改善现有系统的服务效率。已有多智能体研究主要关注前端应用逻辑和交互协议，KVFlow 则说明，工作流语义也能为系统层优化提供重要信息。

参考文献

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [2] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.
- [3] Sirui Hong, Xiwu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 3(4):6, 2023.
- [4] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for” mind” exploration of large language model society. *Advances in Neural Information Processing Systems*, 36:51991–52008, 2023.
- [5] Yiyang Wang, Xiaojing Li, Binzhu Wang, Yueyang Zhou, Yingru Lin, Han Ji, Hong Chen, Jinshi Zhang, Fei Yu, Zewei Zhao, et al. Peer: Expertizing domain-specific tasks with a multi-agent framework and tuning methods. *arXiv preprint arXiv:2407.06985*, 2024.
- [6] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [7] Mingchen Zhuge, Wenqi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.

⁶译注：这里应理解为主要优化对象不同，不能据此概括两者完全没有缓存优化。Parrot 研究动态前缀共享、引擎亲和及共享 KV 内核；Autellix 的多引擎路由也考虑缓存局部性。KVFlow 更具体的区别，是依据未来智能体执行步数安排树节点淘汰和主动预取。

- [8] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.
- [9] Xuchen Pan, Dawei Gao, Yuexiang Xie, Yushuo Chen, Zhewei Wei, Yaliang Li, Bolin Ding, Ji-Rong Wen, and Jingren Zhou. Very large-scale multi-agent simulation in agentscope. *arXiv preprint arXiv:2407.17789*, 2024.
- [10] Zijian He, Reyna Abhyankar, Vikranth Srivatsa, and Yiyang Zhang. Cognify: Supercharging gen-ai workflows with hierarchical autotuning. *arXiv preprint arXiv:2502.08056*, 2025.
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [12] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, 37:62557–62583, 2024.
- [13] NVIDIA. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM>, 2025.
- [14] vLLM Team. Automatic Prefix Caching. https://docs.vllm.ai/en/latest/features/automatic_prefix_caching.html, 2025.
- [15] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*, 2024.
- [16] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, 2024.
- [17] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- [18] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [19] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.
- [20] Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code generation with alphacodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500*, 2024.
- [21] Zhenhailong Wang, Shaoguang Mao, Wenshan Wu, Tao Ge, Furu Wei, and Heng Ji. Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration. *arXiv preprint arXiv:2307.05300*, 2023.
- [22] Yujie Zhao, Hejia Zhang, Hanxian Huang, Zhongming Yu, and Jishen Zhao. Mage: A multi-agent engine for automated rtl code generation. *arXiv preprint arXiv:2412.07822*, 2024.
- [23] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023.
- [24] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first International Conference on Machine Learning*, 2023.
- [25] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

- [26] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.
- [27] Apoorv Saxena. Prompt lookup decoding, November 2023.
- [28] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv*, 2023.
- [29] Yichao Fu, Junda Chen, Siqi Zhu, Zheyu Fu, Zhongdongming Dai, Aurick Qiao, and Hao Zhang. Efficiently serving llm reasoning programs with certaintindex. *arXiv preprint arXiv:2412.20993*, 2024.
- [30] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.
- [31] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.
- [32] Jiachen Liu, Jae-Won Chung, Zhiyu Wu, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. Andes: Defining and enhancing quality-of-experience in llm-based text streaming services. *arXiv preprint arXiv:2404.16283*, 2024.
- [33] Lingfan Yu, Jinkun Lin, and Jinyang Li. Stateful large language model serving with pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 144–158, 2025.
- [34] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. Infercept: Efficient intercept support for augmented large language model inference. *arXiv preprint arXiv:2402.01869*, 2024.
- [35] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.
- [36] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 929–945, 2024.
- [37] LangChain. LangGraph. <https://github.com/langchain-ai/langgraph>, 2025.
- [38] Anthropic. Building effective agents. <https://www.anthropic.com/engineering/building-effective-agents>, 2024.
- [39] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, et al. Agentscope: A flexible yet robust multi-agent platform. *arXiv preprint arXiv:2402.14034*, 2024.
- [40] Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for llm-based multi-agent systems. *arXiv preprint arXiv:2410.02506*, 2024.